TYPESCRIPT

MASTER CHEATSHEET

DETAILED EDITION • FUNDAMENTALS TO ADVANCED • SDET & PLAYWRIGHT

52 structured chapters | runtime internals | code patterns | testing scenarios | interview drills

A practical field guide to TypeScript syntax, type safety, functions, arrays, objects, classes, inheritance, interfaces, modules and automation engineering.

CONTENTS & LEARNING PATH

01 Foundations: console, variables, typing, operators, loops

02 Functions: declarations, expressions, arrows, callbacks, overloads

03 Collections: arrays, iteration, higher-order methods, strings

04 Object-oriented TypeScript: objects, classes, inheritance, interfaces

05 Modules, runtime internals and memory

06 Advanced type guards, async callbacks and SDET mini-projects

07 Practice drills and best-practice checklist

TypeScript Master Cheatsheet

Coverage: Fundamentals → advanced TypeScript → runtime internals → practical SDET examples. Examples assume strict: true and modern ECMAScript. TypeScript checks code before execution; JavaScript executes the emitted code.

1. Console, comments, and setup

```
console.log('Hello TS'); console.info('Info'); console.warn('Warning');
console.error('Failure'); console.table([{id:1, pass:true}]);
// Single-line comment
/* Multi-line comment */
/** JSDoc: documents APIs and supports editor hints. */
```

Install typescript, run npx tsc --init, compile npx tsc file.ts or use a TS runner. console.log writes to a configured output stream; it does not return the logged value.

2. Variables: declaration, rules, scope, hoisting, memory

Variable: named binding to a value. Identifiers are case-sensitive; start with a letter, _, or $ (Unicode identifiers also exist), not a digit; avoid reserved keywords; use meaningful camelCase names.

```
let count: number = 1; count = 2; // mutable binding
const BASE_URL = 'https://example.test'; // must initialize, cannot reassign
var legacy = 'avoid'; // function-scoped, can redeclare
let inferred = 'text'; // inferred string; inferred = 5 => type error
let later: number; later = 3;
const settings = { retries: 1 }; settings.retries = 2; // allowed
// settings = { retries: 3 }; // not allowed
```

Property	var	let	const
Scope	function/global	block	block
Redeclaration in same scope	yes	no	no
Reassignment	yes	yes	no
Hoisting	initialized to undefined	TDZ until declaration	TDZ until declaration
Initialization required	no	no (definite assignment still checked)	yes

Internal model: lexical environments hold bindings; engines decide physical stack/heap layout. Objects/arrays are reference values, but precise allocation is engine-dependent. Closures can retain bindings beyond the creating function. var is initialized at environment creation; let/const bindings exist but cannot be read during the temporal dead zone (TDZ).

```
function scopeDemo() {
  if (true) { var a = 1; let b = 2; const c = 3; }
  console.log(a); // 1; b and c unavailable here
}
// console.log(x); let x = 1; // ReferenceError at runtime
```

Reinitialization vs reassignment: initialization first supplies a value; reassignment replaces it. A second let x declaration in the same scope is illegal. 5 practical bindings: const endpoint, let retryCount, const testData, let token, const selectors.

3. Data types and typing

JavaScript is dynamically typed: values have types, bindings may hold values of different types. TypeScript adds static checks: type annotations, inference, narrowing, structural typing. Static checks are erased at runtime; validate untrusted API responses at runtime.

```
let age: number = 26; let name: string = 'Asha'; let active: boolean = true;
let missing: undefined = undefined; let empty: null = null;
let id: string | number = 'A-12'; id = 12;
function notify(message: string): void { console.log(message); }
function fail(reason: string): never { throw new Error(reason); }
let unknownResponse: unknown = JSON.parse('{"ok":true}');
if (typeof unknownResponse === 'object' && unknownResponse !== null) console.log(unknownResponse);
type Status = 'new' | 'running' | 'done';
interface User { id: number; email: string; active?: boolean; }
const user: User = { id: 1, email: 'qa@example.test' };
```

Categories: JS primitives (string, number, bigint, boolean, symbol, undefined, null); object/reference types (objects, arrays, functions); TS-specific modeling (any, unknown, never, void, unions, intersections, literals, tuples, enums, interfaces, aliases, generics). number covers floating-point and special values NaN/Infinity; bigint is separate. null = intentional absence; undefined = missing/uninitialized value. void normally models ignored return values. any opts out of safety; prefer unknown and narrowing. object excludes primitives; {} means any non-nullish value, not an empty object shape.

When to use what — 5 real examples:

1. string: Salesforce record ID or API bearer token.

2. number: response time in milliseconds or retry counter.

3. boolean: isAuthenticated test assertion flag.

4. string | null: optional access token before login.

5. interface ApiResult<T> { data: T; status: number }: typed API contract.

Type safety: catches incorrect operations during compilation, not malformed server responses. typeof, in, discriminated unions and custom type guards narrow types.

```
type ApiOutcome = { ok: true; data: User } | { ok: false; error: string };
function handle(result: ApiOutcome) {
  if (result.ok) console.log(result.data.email);
  else console.error(result.error);
}
```

4. Operators and evaluation

Family	Operators	Example / result				
Arithmetic	+ - * / % **	7 % 3 === 1, 2 ** 3 === 8				
Assignment	=	let n = 3				
Shorthand	+= -= *= /= %= **=	n += 2				
Relational	< > <= >=	3 < 4				
Loose equality	== !=	0 == false → true (coercion)				
Strict equality	=== !==	0 === false → false				
Logical	`&&		!`	ready && submit()		
Nullish	?? ??=	timeout ?? 5000				
Optional chaining	?.	response.data ?.id				
Logical assignment	`&&=		= ??=`	`token		= 'guest'`
Increment/decrement	++ --	++n, n--				
Ternary	?:	ok ? 'PASS' : 'FAIL'				
Type-related	typeof, instanceof, in	typeof id === 'string'				
Bitwise	`&	^ ~ << >> >>>`	5 & 3 → 1			
Spread/rest syntax	...	[...items], (...args) (syntax, not arithmetic operators)				
TS-only	as, satisfies, !	config satisfies Config (erased at runtime)				

How evaluation works: precedence groups operations, associativity decides ties, operands are evaluated in a specified order, and &&, ||, ??, ?. may short-circuit. + concatenates if string conversion applies; === avoids coercion. ?? defaults only on null/undefined, unlike || which also defaults on 0, false, '', NaN.

```
let n = 5;
console.log(n++); // 5; n becomes 6
console.log(++n); // 7; n is 7
console.log(n--); // 7; n becomes 6
console.log(--n); // 5; n is 5
console.log('5' + 2); // '52'
console.log('5' - 2); // 3
const timeout = 0; console.log(timeout || 5000, timeout ?? 5000); // 5000, 0
```

Use parentheses for readability; avoid mixing ?? with &&/|| without parentheses. Combined example: const verdict = status === 200 && latency < 1000 ? 'PASS' : 'FAIL';.

5. Conditionals and loops

```
const status = 201;
if (status >= 200 && status < 300) console.log('success');
else if (status === 401) console.log('unauthorized');
else console.log('other');
switch (status) { case 200: case 201: console.log('ok'); break; default: console.log('other'); }
for (let i = 0; i < 3; i++) console.log(i);
let retries = 0; while (retries < 3) retries++;
do { retries--; } while (retries > 0);
for (const value of ['a','b']) console.log(value); // values
for (const index in ['a','b']) console.log(index); // string keys
```

break exits a loop/switch; continue skips to next iteration. for...in enumerates keys (including enumerable inherited properties), so prefer for...of for array values. for await...of consumes async iterables. Avoid busy-wait loops; use bounded retries with delays.

6. Named functions (Part 1)

A function declaration binds a reusable function name; parameters are local bindings and returns transfer control/value.

```
function add(a: number, b: number): number { return a + b; }
function logAll(...messages: string[]): void { messages.forEach(console.log); }
function lookup(id: number | string): string { return String(id); }
function greet(name: string, title?: string): string { return `${title ?? 'Hi'} ${name}`; }
function retry(maxAttempts: number = 3): number { return maxAttempts; }
```

Internal flow: caller evaluates arguments → creates activation/parameter bindings → executes body → returns value (or undefined) → activation may be released unless closures retain it. Default parameters run when an argument is undefined, not null. Rest parameters collect remaining arguments into an array; optional parameters are modeled as potentially undefined. Function declarations are available before their declaration within their scope (subject to module/block semantics).

7. Anonymous functions (Part 2)

Anonymous functions are function expressions without a declared name, often assigned or passed as callbacks.

```
const multiply = function(a: number, b: number): number { return a * b; };
const tasks = ['login', 'search'];
tasks.forEach(function(task) { console.log(task); });
const makeCounter = function(start = 0) {
  let count = start;
  return function() { return ++count; }; // closure
};
const next = makeCounter(10); next(); // 11
```

Function expressions are evaluated when execution reaches them; accessing a const binding before initialization throws. function expressions have their own dynamic this and arguments. A named function expression (const f = function internal() {}) supports self-reference in its body.

8. Arrow functions (Part 3)

```
const square = (n: number): number => n * n; // implicit return
const objectFactory = (id: number) => ({ id }); // parentheses for object literal
const normalize = (s: string) => s.trim().toLowerCase();
const createPredicate = (min: number) => (n: number) => n >= min;
const delay = (ms: number): Promise<void> => new Promise(resolve => setTimeout(resolve, ms));
```

Arrows capture lexical this and lack their own arguments, prototype, and constructor capability; cannot be called with new. Use rest parameters instead of arguments. async (x) => await work(x) is valid; arrows cannot be generator functions.

9. Named vs anonymous vs arrow (Part 4)

Scenario	Preferred	Why / caution
Exported domain operation	named declaration	discoverable stack traces, hoisting
Short map/filter callback	arrow	concise lexical binding
Object method using dynamic this	method or function expression	arrows do not rebind this
Class callback capturing instance	arrow property or bound method	preserves instance this
Recursive local helper	named function	clear self-reference
Framework hook with context-dependent this	normal function when required	arrow may break framework contract

Web: arrow event callbacks; named validateForm. Unit testing: test('adds', () => expect(add(1,2)).toBe(3)). Functional UI: page.on('dialog', dialog => dialog.accept()). Nonfunctional: named measureLatency and arrow mapping percentiles. API: arrow response transformation, named validateSchema. DB: named transaction helper and arrow result mapping. Choose for semantics, not perceived speed.

10–11. Callbacks (Parts 6–7)

A callback is a function supplied to another function to run later or immediately. It may be synchronous, asynchronous, or event-driven.

```
function process(values: number[], callback: (n: number) => number): number[] {
  return values.map(callback);
}
console.log(process([1,2], n => n * 2)); // [2,4]
function onComplete(done: (err: Error | null, result?: string) => void) {
  setTimeout(() => done(null, 'finished'), 10);
}
onComplete((err, result) => { if (err) throw err; console.log(result); });
```

Internal flow: callback reference passed as value → outer function stores or invokes it → synchronous callback executes on current call stack; timer/event callbacks are scheduled via runtime queues and event loop when eligible. Microtasks (e.g., promise reactions) are processed according to runtime event-loop rules; TS itself does not schedule callbacks.

Advanced scenarios: retry with predicate callback; array transformations; UI event listeners; test hooks; Node error-first callbacks; observer/subscription cleanup; streaming data processors; rate-limit/throttle/debounce. Prefer Promise/async for sequential async workflows and use try/finally for cleanup.

```
async function retryUntil<T>(operation: () => Promise<T>, accepts: (value: T) => boolean, tries = 3):
Promise<T> {
  let lastError: unknown;
  for (let i = 0; i < tries; i++) {
    try { const result = await operation(); if (accepts(result)) return result; }
    catch (error) { lastError = error; }
  }
  throw lastError ?? new Error('Predicate never matched');
}
```

Examples: poll job status, wait for API health, attach screenshot-on-failure callback, filter audit events, handle paginated API results.

12–14. Overloading and overriding (Parts 8–10)

Overloading: multiple compile-time call signatures with one runtime implementation. Overriding: subclass provides replacement behavior for inherited method. TS does not dispatch among overload implementations by signature; the single implementation branches at runtime.

```
function parseId(value: number): string;
function parseId(value: string): string;
function parseId(value: string | number): string { return String(value).trim(); }
parseId(12); parseId(' 12 ');

function findRecord(id: number): string;
function findRecord(id: number, includeDeleted: boolean): string;
function findRecord(id: number, includeDeleted = false): string {
  return `${id}:${includeDeleted}`;
}
class BaseReporter { report(): string { return 'base'; } }
class HtmlReporter extends BaseReporter { override report(): string { return 'html'; } }
```

Constructor overloads: overload declarations precede exactly one constructor body. Method overloads: same pattern inside a class. Optional parameters can replace overloads when return type does not change; use overloads when input shapes produce meaningfully distinct output types. Overloads are ordered; specific signatures should come before broad signatures. Implementation signature is not callable externally unless separately exposed.

```
class Request {
  url: string; timeout: number;
  constructor(url: string);
  constructor(url: string, timeout: number);
  constructor(url: string, timeout = 5000) { this.url = url; this.timeout = timeout; }
  send(body: string): string;
  send(body: object): string;
  send(body: string | object): string { return typeof body === 'string' ? body : JSON.stringify(body);
 }
}
```

Programming exercises with answers: overload format(number): string / format(Date): string; getValue(key:'id'):number / getValue(key:'name'):string; assertEqual(string,string) and (number,number); overload waitFor(selector:string) and waitFor(predicate:()=>boolean); constructor with url and optional timeout. In all cases write one implementation that narrows input safely.

15–16. Arrays (Parts 1–2)

Arrays are ordered, zero-indexed, resizable objects; elements can be heterogeneous at runtime but TS can constrain types. Arrays are reference values and may be sparse.

```
const ids: number[] = [10,20,30];
const names: Array<string> = ['Ada','Lin'];
const pair: readonly [string, number] = ['id', 42]; // tuple
const immutable: readonly number[] = [1,2];
console.log(ids[0]); ids.push(40); ids.pop(); ids.unshift(5); ids.shift();
ids.slice(0,2); ids.splice(1,1); ids.includes(20); ids.indexOf(30);
ids.find(x => x > 10); ids.findIndex(x => x > 10);
ids.sort((a,b) => a-b); ids.reverse(); ids.join(',');
for (let i=0; i<ids.length; i++) console.log(ids[i]);
for (const index in ids) console.log(index); // keys, not values
for (const value of ids) console.log(value);
function total(items: readonly number[]): number { return items.reduce((a,b) => a+b, 0); }
function positives(items: readonly number[]): number[] { return items.filter(x => x>0); }
```

sort and reverse mutate the original array; slice does not, splice does. readonly is compile-time only, not runtime freezing. ids[100] can yield undefined; enable noUncheckedIndexedAccess for stricter indexing.

17. Array higher-order methods

Method	Returns	When to use	Avoid when
forEach(fn)	void	side effects/logging	building transformed arrays, async await coordination
map(fn)	new array	1:1 transformation	ignoring result
filter(fn)	new array	subset selection	only checking existence (some)
reduce(fn, init)	accumulator	aggregations/grouping	a simple loop is clearer
some(fn)	boolean	at least one matches	all must match (every)
every(fn)	boolean	all match	finding matching item (find)
sum()	not a standard Array method	use reduce	assuming built-in exists

```
const results = [{name:'A', ms:100, pass:true}, {name:'B', ms:400, pass:false}];
results.forEach(r => console.log(r.name));
const times = results.map(r => r.ms); // [100,400]
const failed = results.filter(r => !r.pass);
const sum = results.reduce((acc,r) => acc + r.ms, 0); // 500
const anyFailed = results.some(r => !r.pass); // true
const allPassed = results.every(r => r.pass); // false
const average = results.length ? sum / results.length : 0;
```

Optimization: each map/filter traverses input and allocates output; combine passes only when profiling shows a benefit. reduce with {} spread inside each iteration can be expensive; mutate a local accumulator when appropriate. forEach(async x => ...) does not await tasks: use for...of + await sequentially, or await Promise.all(items.map(async ...)) concurrently (consider limits).

18. Strings

Strings are immutable UTF-16 sequences of code units; indexing is zero-based, and surrogate pairs can occupy two indices. JS engines choose internal representations; concatenation may create ropes and flatten later, so avoid assuming stack/heap layout.

```
const s: string = '  Hello TypeScript  ';
s.length; s[2]; s.charAt(2); s.at(-1);
s.trim(); s.trimStart(); s.trimEnd(); s.toLowerCase(); s.toUpperCase();
s.includes('Type'); s.startsWith('  He'); s.endsWith('  ');
s.indexOf('Type'); s.lastIndexOf('e');
s.slice(2,7); s.substring(2,7); // substring swaps reversed arguments; slice does not
s.replace('Hello','Hi'); s.replaceAll(' ', '_');
s.split(' '); s.concat('!'); s.padStart(25,'.'); s.repeat(2);
'abc123'.match(/\d+/); /^\S+@\S+\.\S+$/.test('a@b.co');
const multiline = `Line one\nLine two\n${s.trim()}`;
```

Validation: regex for format, explicit length/range checks, and schema validation for real inputs. replace with a string replaces first occurrence only; replaceAll replaces all. slice accepts negative indices. for...of iterates Unicode code points, unlike numeric indexing. Examples: trim input, normalize email, extract record prefix, mask account numbers, validate response headers.

19–21. Objects and object types

Objects group keyed properties and methods. Keys are strings or symbols (numeric property keys are converted to strings). Objects use reference identity.

```
const account = { id: 1, name: 'Demo', display() { return this.name; } };
account.name; account['name']; account.display(); account.name = 'Updated';
const key: keyof typeof account = 'name'; console.log(account[key]);
const copy = { ...account }; // shallow copy, not deep
const fromFactory = Object.create(null); // null prototype
const viaConstructor = new Object();
type Identified = { id: number };
type Contact = { email: string };
type Customer = Identified & Contact;
const customer: Customer = { id: 2, email: 'x@example.test' };
const inline: { label: string; enabled?: boolean } = { label: 'QA' };
```

type can model unions, primitives, tuples and intersections; interface can extend and merge declarations. Object.keys, Object.values, Object.entries, Object.assign, Object.hasOwn are common utilities. delete obj.key removes an optional property; prefer creating a new object when immutable updates are needed. Two objects with equal fields are still different references: {a:1} === {a:1} is false. A spread copy shares nested object references.

22–28. Classes (Parts 1–7)

Classes are templates for objects, with instance properties/methods and static members. Runtime classes are JS constructor/prototype mechanisms; TypeScript visibility and readonly are normally compile-time restrictions.

```
class TestCase {
  static created = 0; // shared by class
  readonly id: string;
  description?: string;
  private attempts = 0; // TS private: compile-time only
  #secret = 'internal'; // JS runtime-private field
  constructor(id: string, description?: string) {
    this.id = id; this.description = description; TestCase.created++;
  }
  run(): string { this.attempts++; return `${this.id}: ${this.description ?? 'unnamed'}`; }
  display = (): string => this.run(); // arrow instance field
  static getCreated(): number { return TestCase.created; }
}
const a = new TestCase('TC-01'); const b = new TestCase('TC-02','Login');
a.description = 'Smoke'; // independent instance state
console.log(a.run(), b.run(), TestCase.getCreated());
// a.id = 'new'; // compile-time readonly error
```

Internal diagram: TestCase constructor + prototype methods (run) → new creates distinct instances (a, b) → each instance owns its fields (id, description, attempts, display) → both inherit run from prototype → static created belongs to constructor object. Instance arrow fields allocate a function per instance; prototype methods are shared. readonly does not freeze the JS value at runtime. #secret is enforced by JS runtime.

Types of classes: concrete, abstract, base, derived; TS has no final class keyword and no built-in sealed-class enforcement. Abstract classes cannot be instantiated directly by TS-checked code and may define abstract methods. Constructors may be overloaded (one implementation), but cannot be async. Static members cannot directly access instance fields without an instance. 5 use cases: Page Object, API client, test data builder, report aggregator, configuration singleton-like utility (prefer simple modules where suitable).

```
abstract class BasePage {
  constructor(protected readonly path: string) {}
  abstract title(): string;
  url(base: string): string { return base + this.path; }
}
class LoginPage extends BasePage {
  constructor() { super('/login'); }
  override title(): string { return 'Sign in'; }
}
```

29–33. Inheritance (Parts 1–5)

Inheritance reuses/extends behavior via extends. TS/JS classes support single class inheritance; classes may implement multiple interfaces. Interfaces can extend multiple interfaces. Java supports single class inheritance and multiple interfaces as well. JS prototype chains underpin TS class inheritance at runtime.

```
class Vehicle {
  constructor(public brand: string) {}
  start(): string { return `${this.brand} started`; }
}
class Car extends Vehicle {
  constructor(brand: string, public seats: number) { super(brand); }
  override start(): string { return super.start() + ' (car)'; }
}
const vehicle: Vehicle = new Car('Demo',4);
console.log(vehicle.start()); // dynamic dispatch invokes Car.start
```

Patterns: single (Car extends Vehicle), multilevel (ElectricCar extends Car), hierarchical (Car and Bike extend Vehicle); multiple class inheritance is not supported directly. Composition and mixins can model combined behavior. Access: public accessible everywhere; protected within class/subclasses; private only declaring class (TS checks); JS #private runtime enforced. A derived constructor must call super() before accessing this.

Diagram: Car instance → Car.prototype → Vehicle.prototype → Object.prototype → null. Property lookup walks the chain until found. Overridden methods shadow parent methods; super.start() explicitly calls the parent implementation. The type of a variable (Vehicle) controls compile-time access; actual instance (Car) controls overridden runtime behavior.

5 practical inheritance scenarios: base Playwright page → login page; base API client → Salesforce API client; base test fixture → specialized fixture; generic report → HTML report; common domain exception → API exception. Avoid deep hierarchies and inheritance used only to share a few helpers; prefer composition.

34–37. Interfaces (Parts 1–4)

An interface describes a structural contract for object shapes. It does not produce a JavaScript runtime object, constructor, or validator.

```
interface HasId { readonly id: string; }
interface HasName { name?: string; }
interface Employee extends HasId, HasName {
  role: 'qa' | 'dev';
  describe(): string;
}
class Engineer implements Employee {
  constructor(public readonly id: string, public role: 'qa' | 'dev', public name?: string) {}
  describe(): string { return `${this.id}: ${this.role}`; }
}
interface ApiResponse<T> { status: number; data: T; }
interface Formatter { (value: string): string; }
interface Dictionary { [key: string]: number; }
```

Kinds of interface shapes: object, callable, construct signature, indexable, generic, extending, hybrid. Interfaces support declaration merging; type aliases do not. implements is compile-time verification only and does not copy code. Interface vs class: interface = shape/contract, erased at runtime; class = runtime constructor, instances, implementation and inheritance. Interface vs type alias: use either for object shapes; type is needed for unions/conditional/mapped types and convenient for intersections. readonly/optional are checked statically, not runtime locks.

38. Modules

A file with top-level import/export is a module; top-level declarations are scoped to that module rather than becoming globals. Named/default exports, re-exports, type-only imports, and dynamic imports are available.

```
// math.ts
export const PI = 3.14159;
export function add(a: number, b: number) { return a+b; }
export type Result = { ok: boolean };
// main.ts
import { PI, add } from './math.js';
import type { Result } from './math.js';
const answer: Result = { ok: add(1,2) > PI };
// const loaded = await import('./math.js'); // dynamic import
```

Types: ES modules (ESM), CommonJS (Node legacy), ambient declarations (.d.ts), namespaces (legacy organizational feature, not modern module replacement). Internal flow: resolver finds module → loader links imports/exports → module graph evaluates dependencies → import bindings are live read-only views. ESM modules are evaluated once per resolved module identity within the relevant loader cache; circular imports require care with initialization order. TS compiler module and moduleResolution must match runtime/bundler (e.g., NodeNext for Node ESM). Relative .js specifiers are often appropriate in TS source for Node ESM emitted JS.

Practical Playwright + API testing mini-project

```
import { test, expect, type APIRequestContext, type Page } from '@playwright/test';
interface LoginUser { username: string; password: string; }
class LoginPage {
  constructor(private readonly page: Page) {}
  async login(user: LoginUser): Promise<void> {
    await this.page.goto('/login');
    await this.page.getByLabel('Username').fill(user.username);
    await this.page.getByLabel('Password').fill(user.password);
    await this.page.getByRole('button', { name: 'Sign in' }).click();
  }
}
async function fetchStatus(request: APIRequestContext): Promise<number> {
  const response = await request.get('/api/health');
  return response.status();
}
test('login and API health', async ({ page, request }) => {
  const login = new LoginPage(page);
  await login.login({ username: 'demo', password: 'test-secret' });
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
  expect(await fetchStatus(request)).toBe(200);
});
```

Replace sample routes, labels, credentials and assertions for your app; never hardcode real secrets. For DB tests, use parameterized queries, transactions/rollback, and typed result mapping. For load tests, use a dedicated load-testing tool rather than assuming a Playwright UI loop measures capacity accurately.

Common mistakes and interview quick checks

1. TS type annotations do not validate JSON at runtime.
2. const prevents rebinding, not mutation of referenced objects.
3. let/const are hoisted into the TDZ; they are not simply “not hoisted.”
4. == coerces; prefer === unless coercion is intentional.
5. null differs from undefined; ?? differs from ||.
6. typeof null is 'object' (historic JS behavior).
7. NaN === NaN is false; use Number.isNaN.
8. Array.sort() sorts lexicographically by default; pass numeric comparator.
9. forEach does not await asynchronous callbacks.
10. Arrow functions capture lexical this; class prototype methods do not.
11. Overloads are erased and have one runtime implementation.
12. Overriding is runtime method dispatch through the prototype chain.
13. Interfaces and type aliases are erased; classes remain in JavaScript.
14. Static properties live on constructors, not ordinary instances.
15. Object spread and array spread make shallow copies.
16. string.length counts UTF-16 code units, not necessarily user-perceived characters.
17. private (TS) is different from #private (JS).
18. readonly is compile-time protection, not Object.freeze.

19. unknown requires narrowing; any bypasses checks.

20. Prefer noImplicitAny, strictNullChecks, noUncheckedIndexedAccess where feasible.

Suggested compiler configuration

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  }
}
```

Practice path: implement typed test data → reusable utility functions → array processing/report summaries → API client with interfaces → Page Object classes → inheritance/composition → overloaded helpers → modular Playwright project.

39. Deep dive: runtime memory, closures and garbage collection

Mental model (not a guaranteed physical layout): source code is parsed, lexical environments are created, bindings resolve by scope chain, and values are stored in engine-managed memory. A primitive assignment copies the primitive value; assigning an object shares its reference. Engines may optimize allocation and representation.

```
let first = { count: 1 };
let second = first;
second.count = 9;
console.log(first.count); // 9: same object
second = { count: 0 }; // now points to another object
console.log(first.count); // 9
function makeIdGenerator() {
  let nextId = 0;
  return () => ++nextId; // captured binding survives return
}
const nextId = makeIdGenerator();
console.log(nextId(), nextId()); // 1, 2
```

Diagram: first ─┐ [object count:9] ┌─ second (before reassignment); after reassignment, second → [object count:0]. Reachability determines whether objects can be garbage-collected. Avoid treating 'stack vs heap' as a precise language guarantee.

Testing use: create isolated fixtures per test; avoid mutable module-level state that leaks between test cases.

40. Deep dive: type narrowing, guards and discriminated unions

```
type Success<T> = { ok: true; status: 200; data: T };
type Failure = { ok: false; status: number; error: string };
type Result<T> = Success<T> | Failure;
function formatResult(result: Result<{ id: string }>): string {
  if (result.ok) return result.data.id;
  return `HTTP ${result.status}: ${result.error}`;
}
function isRecord(x: unknown): x is Record<string, unknown> {
  return typeof x === 'object' && x !== null && !Array.isArray(x);
}
function hasStringId(x: unknown): x is { id: string } {
  return isRecord(x) && typeof x.id === 'string';
}
const payload: unknown = JSON.parse('{"id":"A001"}');
if (hasStringId(payload)) console.log(payload.id.toUpperCase());
```

Internal working: TypeScript control-flow analysis narrows static types; JavaScript executes the actual checks. Type guards are not generated automatically from interfaces. Use cases: API schema validation, Salesforce record responses, fixture parsing, environment configuration, and database query results.

41. Deep dive: all common operators with evaluation traps

```
let x = 2;
const a = x++ + ++x; // 2 + 4 = 6; x = 4
const b = 0 || 10;   // 10 (falsy fallback)
const c = 0 ?? 10;   // 0 (nullish fallback)
const d = false && expensive(); // does not call expensive
const e = true || expensive();  // does not call expensive
const f = ({user: {name: 'A'}}).user?.name ?? 'Guest';
const g = 2 + 3 * 4; // 14, multiplication first
const h = (2 + 3) * 4; // 20
```

Evaluation order: JavaScript evaluates operands left-to-right; operator precedence determines grouping, not an arbitrary reordering of side effects. == coerces types, === generally does not; object equality compares identity. Avoid side effects inside complex expressions. ?. only guards the immediately chained access and does not validate response shapes.

42. Deep dive: iteration and algorithmic trade-offs

```
const rows = [{id: 1, pass: true}, {id: 2, pass: false}];
rows.forEach(row => console.log(row.id)); // side effects
const ids = rows.map(row => row.id); // [1,2]
const failures = rows.filter(row => !row.pass); // subset
const passed = rows.every(row => row.pass); // false
const anyFailed = rows.some(row => !row.pass); // true
const count = rows.reduce((total, row) => total + Number(row.pass), 0);
const sum = [10, 20, 30].reduce((acc, n) => acc + n, 0);
```

Choice: forEach for synchronous side effects (does not await promises); map for one-to-one transforms; filter for selection; reduce for accumulation; some/every for predicates with short-circuiting. Standard arrays do not expose sum() in JavaScript. Most of these scan O(n) items and map/filter allocate a new array. for...of with await is preferable for sequential async steps.

43. Deep dive: async callbacks and Playwright pitfalls

```
// BAD: forEach does not await asynchronous callbacks
// urls.forEach(async url => await page.goto(url));
// Sequential navigation:
for (const url of urls) {
  await page.goto(url);
}
// Independent API requests may run concurrently:
const results = await Promise.all(
  urls.map(url => request.get(url))
);
for (const response of results) {
  if (!response.ok()) throw new Error(`HTTP ${response.status()}`);
}
```

Internal working: async functions return promises; await suspends the async continuation without blocking the JavaScript thread. Promise.all rejects when an input rejects but does not cancel other in-flight work. Testing: parallelize independent API reads, not shared-page UI navigation; bound concurrency to protect environments.

44. Deep dive: function overload signatures and implementation

```
function readValue(key: 'timeout'): number;
function readValue(key: 'baseUrl'): string;
function readValue(key: 'timeout' | 'baseUrl'): number | string {
  return key === 'timeout' ? 5000 : 'https://example.test';
}
const timeout = readValue('timeout'); // number
const baseUrl = readValue('baseUrl'); // string
```

Rule: overload signatures are compile-time declarations; only the implementation function exists at runtime. Put specific signatures before broad ones; ensure the implementation handles every advertised signature. Optional parameters are simpler when return types do not depend on argument shape. Overriding is different: a subclass provides a method implementation compatible with the base class contract; use override for clarity.

45. Deep dive: constructors, static state and inheritance

```
abstract class BaseClient {
  static instances = 0;
  constructor(protected readonly baseUrl: string) {
    BaseClient.instances++;
  }
  protected path(route: string): string { return this.baseUrl + route; }
  abstract healthPath(): string;
}
class ApiClient extends BaseClient {
  constructor(url: string, private readonly token?: string) { super(url); }
  override healthPath(): string { return this.path('/health'); }
  getToken(): string | undefined { return this.token; }
}
const clientA = new ApiClient('https://api.example.test');
const clientB = new ApiClient('https://api.example.test', 'secret');
console.log(clientA.healthPath(), BaseClient.instances); // path, 2
```

Internal working: new creates an instance linked to the class prototype, initializes base and derived instance fields according to JavaScript class rules, then returns the instance. Static members belong to the constructor, not each instance. private and protected in TypeScript are normally compile-time restrictions; JavaScript #private is enforced at runtime. TypeScript does not have a built-in final class keyword.

46. Deep dive: interfaces, aliases and structural compatibility

```
interface HasId { id: string }
interface HasName { name: string }
interface Account extends HasId, HasName { active?: boolean }
type Audited = { createdAt: Date };
type AuditedAccount = Account & Audited;
const account: AuditedAccount = {
  id: '001', name: 'Acme', createdAt: new Date()
};
interface Repository<T extends HasId> {
  findById(id: string): Promise<T | null>;
  save(entity: T): Promise<void>;
}
```

Interfaces can be declaration-merged and extended; type aliases can express unions, primitives, tuples and mapped types. Both are erased at runtime. Structural typing checks the members present, not nominal identity. For contracts, prefer interfaces; for unions and transformations, type aliases are often clearer.

47. Deep dive: modules, ESM and dependency boundaries

```
// math.ts
export const PI = 3.14159;
export function area(radius: number): number { return PI * radius ** 2; }
// app.ts
import { area } from './math.js'; // NodeNext ESM emitted-path style
console.log(area(2));
// types.ts
export interface TestCase { id: string; expectedStatus: number }
// consumer.ts
import type { TestCase } from './types.js';
```

Runtime model: ECMAScript modules are resolved, linked, then evaluated; imports are live bindings. import type disappears in emitted JS. Circular imports can trigger initialization/TDZ errors. CommonJS uses require and module.exports; interop depends on compiler and runtime settings. Choose module/moduleResolution settings for the actual Node or bundler environment.

48. Practical SDET mini-project: typed API assertions

```
import { test, expect } from '@playwright/test';
interface Account { id: string; name: string }
function isAccount(x: unknown): x is Account {
  if (typeof x !== 'object' || x === null) return false;
  const obj = x as Record<string, unknown>;
  return typeof obj.id === 'string' && typeof obj.name === 'string';
}
test('account API returns a typed record', async ({ request }) => {
  const response = await request.get('/api/accounts/001');
  expect(response.status()).toBe(200);
  const body: unknown = await response.json();
  expect(isAccount(body)).toBe(true);
  if (!isAccount(body)) throw new Error('Unexpected account schema');
  expect(body.id).toBe('001');
});
```

Why this matters: response.json() should be treated as untrusted data; an interface alone cannot validate it. This sample requires a configured Playwright baseURL and a matching test endpoint.

49. Practical SDET mini-project: reusable retry helper

```
type RetryOptions = { attempts: number; delayMs: number };
const sleep = (ms: number) => new Promise<void>(resolve => setTimeout(resolve, ms));
async function retry<T>(task: () => Promise<T>, opts: RetryOptions): Promise<T> {
  if (opts.attempts < 1) throw new Error('attempts must be positive');
  let lastError: unknown;
  for (let attempt = 1; attempt <= opts.attempts; attempt++) {
    try { return await task(); }
    catch (error) {
      lastError = error;
      if (attempt < opts.attempts) await sleep(opts.delayMs);
    }
  }
  throw lastError;
}
```

When to use: transient idempotent API operations and polling. When not to use: hiding deterministic assertion failures, repeating non-idempotent POSTs without safeguards, or replacing Playwright's built-in auto-waiting. Add exponential backoff and jitter when retrying shared services.

50. Practical SDET mini-project: SQL-style data transformation

```
type DbRow = { account_id: string; amount: number | null };
type Summary = { accountId: string; total: number };
function summarize(rows: readonly DbRow[]): Summary[] {
  const totals = new Map<string, number>();
  for (const row of rows) {
    totals.set(row.account_id, (totals.get(row.account_id) ?? 0) + (row.amount ?? 0));
  }
  return [...totals].map(([accountId, total]) => ({ accountId, total }));
}
console.log(summarize([
  { account_id: 'A', amount: 20 },
  { account_id: 'A', amount: null },
  { account_id: 'B', amount: 15 }
]));
```

Testing: compare API totals with independently queried DB aggregates; normalize ordering and null-handling first. Use parameterized SQL for actual database calls; do not concatenate user input into queries.

51. Practice drills and interview checks

1. Explain why typeof null returns 'object' and why a null check is necessary.
2. Predict output of let x=3; console.log(x++, ++x); justify evaluation order.
3. Explain why const arr=[]; arr.push(1) works while arr=[] fails.
4. Rewrite items.forEach(async ...) using sequential for...of.
5. Explain why interface User does not validate JSON.parse() output.
6. Implement unique<T>(items: T[]): T[] using Set.
7. Explain the difference between unknown, any, and never.
8. Implement a discriminated union for success and failure API responses.
9. Compare method overload signatures with subclass method overriding.
10. Explain how super() initializes a derived class instance.
11. Explain ?? versus || when a timeout is zero.

12. Explain the difference between private and JavaScript #private.

13. Explain why Array.prototype.sum() cannot be assumed to exist.

14. Identify when Promise.all is unsafe for UI navigation.

15. Describe the runtime difference between import and import type.

52. Quick reference: best practices checklist

- Enable strict, noUncheckedIndexedAccess where practical, and meaningful lint rules.

- Prefer const by default, let for reassignment, and avoid var in new code.

- Prefer unknown for untrusted data and validate at system boundaries.

- Use discriminated unions for mutually exclusive result states.

- Prefer === and !== except when deliberate coercion is documented.

- Use ?? for nullish defaults and ?. for optional access.

- Use for...of for sequential asynchronous operations.

- Prefer readable named functions for exported logic; use arrows for concise callbacks.

- Do not confuse TypeScript type checking with runtime validation.

- Keep test fixtures independent and shared static state under control.

- Model external APIs with interfaces plus runtime validators.

- Keep module dependencies acyclic where possible.